

Building LLM - Relazione generale del progetto

Introduzione

Building LLM è un progetto didattico che parte dal modello linguistico più semplice e comprensibile e procede, una versione alla volta, verso uno small LLM moderno.

Ogni versione introduce un concetto mirato, mantenendo i meccanismi visibili, eseguibili e adatti allo studio.

una versione -> un concetto -> un risultato verificabile

Obiettivo del progetto

L'obiettivo non è competere con modelli industriali. Lo scopo è comprendere come vengono costruiti, addestrati, valutati e utilizzati i modelli linguistici, passando dai conteggi statistici a un Transformer decoder-only.

Il progetto segue questi principi:

- l'inglese è la lingua iniziale del modello;
 - i primi meccanismi sono implementati dalle basi;
 - la complessità aumenta gradualmente e resta spiegabile;
 - i notebook sono eseguibili dall'alto verso il basso;
 - la riproducibilità conta più della scala del modello;
 - il training avanzato è pensato per hardware Kaggle accessibile.
-

Vincoli del progetto

- la Versione 1 usa esclusivamente Python standard;
- NumPy viene introdotto prima di un framework di machine learning;
- TensorFlow appare solo dopo aver compreso una piccola rete neurale;
- i dataset pubblici richiedono origine, versione e licenza documentate;
- ogni versione conserva un controllo rapido che non richiede una GPU.

Che cosa significa LLM in questo percorso

Le prime versioni sono modelli linguistici, ma non sono ancora large language model. Il termine LLM diventa appropriato solo dopo l'introduzione della tokenizzazione subword, di un Transformer decoder-only, di un numero significativo di parametri e del training su un corpus sostanziale.

Mantenere esplicita questa distinzione evita di presentare un piccolo modello statistico come qualcosa di più grande.

Il ciclo stabile del modello linguistico

Nel progetto rimane stabile il seguente ciclo concettuale:

```
testo -> rappresentazione -> contesto
      -> probabilità del simbolo successivo
      -> campionamento -> testo generato
```

La Versione 1 ottiene le probabilità direttamente dai conteggi dei caratteri osservati. Le versioni successive introducono parametri addestrabili, obiettivi di apprendimento, ottimizzazione, contesti più ricchi e nuove rappresentazioni dei token.

La vera base della Versione 1

La Versione 1 è un modello linguistico statistico a caratteri scritto con Python standard. Legge un corpus inglese incorporato, crea coppie di caratteri adiacenti, conta le transizioni, normalizza i conteggi in probabilità e campiona un carattere alla volta.

I controlli implementati verificano:

- il numero corretto di esempi formati da caratteri adiacenti;
- la somma delle probabilità del carattere successivo uguale a 1;
- la riproducibilità della generazione con lo stesso seed;
- la lunghezza richiesta dell'output generato.

La Versione 1 usa soltanto conteggi osservati, normalizzazione delle probabilità e campionamento. Non contiene componenti neurali addestrabili.

Roadmap - Fondamenti statistici e neurali

Versione 1 - Modello statistico a caratteri

Conta le transizioni tra caratteri adiacenti, converte i conteggi in probabilità e genera testo con sampling riproducibile usando Python standard.

Versione 2 - Piccola rete neurale con NumPy

Sostituisce la tabella dei conteggi con un piccolo modello neurale e rende visibile l'apprendimento dei parametri tramite operazioni NumPy.

Versione 3 - Fondamenti del training

Introduce batch, suddivisione dei dati, un obiettivo di training, ottimizzazione e riproducibilità sistematica.

Versione 4 - Introduzione a TensorFlow

Ricostruisce il piccolo modello con TensorFlow e differenziazione automatica dopo averne compreso i calcoli fondamentali.

Versione 5 - Embedding e finestre di contesto

Apprende rappresentazioni vettoriali e usa più di un carattere precedente come contesto.

Roadmap - Sequenze e attention

Versione 6 - Modello linguistico ricorrente

Introduce una RNN semplice e uno stato nascosto che trasporta informazioni nella sequenza.

Versione 7 - Modello GRU o LSTM

Migliora l'apprendimento delle sequenze e la stabilità del training con unità ricorrenti dotate di gate.

Versione 8 - Attention

Costruisce la scaled dot-product attention da componenti comprensibili e mostra come i token scambiano informazioni.

Versione 9 - Blocco Transformer

Combina self-attention causale, livelli feed-forward, connessioni residue e normalizzazione.

Versione 10 - Mini modello decoder-only

Impila blocchi Transformer e realizza la generazione autoregressiva con mascheramento causale.

Roadmap - Dati, training e utilizzo

Versione 11 - Tokenizer subword e dataset pubblici

Passa dai caratteri ai token subword e introduce dataset aperti documentati.

Versione 12 - Pipeline di training Kaggle

Aggiunge pipeline efficienti, checkpoint, ripresa del training e mixed precision quando l'hardware disponibile la supporta.

Versione 13 - Valutazione e generazione controllata

Introduce una valutazione adatta al modello addestrato e controlli di generazione come temperatura, top-k e top-p.

Versione 14 - Instruction tuning

Esegue il fine-tuning su piccoli dataset di istruzioni con licenze compatibili con il flusso previsto.

Versione 15 - Small LLM avanzato

Organizza configurazione, training, inferenza, documentazione, model card ed esperimenti riproducibili.

Strategia per i dataset

La Versione 1 incorpora direttamente nel notebook un piccolo corpus didattico. Le versioni successive possono passare prima a piccoli file di testo e poi a corpus pubblici come Tiny Shakespeare, WikiText o dataset didattici equivalenti.

Prima del training su un dataset reale, il progetto documenta:

- origine e versione esatta del dataset;
 - licenza e utilizzi consentiti;
 - lingua e ambito dei contenuti;
 - decisioni di pulizia e preprocessing;
 - suddivisioni di training, validation e test.
-

Strategia Kaggle

- notebook eseguibili dall'alto verso il basso;
 - seed e configurazioni espliciti;
 - percorsi dei dataset configurabili sotto `/kaggle/input`;
 - checkpoint delle versioni avanzate salvati sotto `/kaggle/working`;
 - configurazioni CPU e GPU con dimensioni conservative;
 - un percorso di verifica rapido disponibile senza GPU.
-

Qualità, sicurezza e documentazione

Ogni versione include controlli adeguati al proprio ambito, limitazioni esplicite e una chiara gestione degli errori. Le suddivisioni dei dati e le metriche quantitative vengono aggiunte quando il modello e la fase di training le rendono applicabili.

Le versioni che usano dataset reali includono note su provenienza e licenza. Le versioni con modelli addestrati includono istruzioni riproducibili e una model card con utilizzi previsti e limitazioni.

Risultato finale atteso

Il risultato atteso è uno small LLM inglese decoder-only che possa essere addestrato o sottoposto a fine-tuning su hardware Kaggle accessibile, con tokenizer, pipeline TensorFlow, checkpoint, generazione controllata, valutazione adeguata e documentazione. Il progetto resta centrato sulla comprensione e sulla riproducibilità, non sulla scala dei modelli commerciali.